

Trabajo Práctico 3

Procesamiento de Imágenes y Visión por Computadora

Matías Cisnero

6 de Noviembre de 2025

Consigna 1:

Implementar el detector de bordes de Canny. Aplicarlo a una imagen y a su versión contaminada con distintos tipo de ruido.



Detector de Canny

- ➊ Aplicar un filtro Gaussiano.
- ➋ Aplicar el método de Sobel para calcular aproximadamente el gradiente y con esto la Magnitud de Borde.
- ➌ Calcular el ángulo del gradiente.
- ➍ Discretizar los ángulos en 4 valores (0° , 45° , 90° , 135°).
- ➎ Realizar la supresión de no máximos.
- ➏ Aplicar umbralización con histéresis.

(*) Obviaremos el paso 1 y 2 porque ya lo hemos realizado anteriormente.



El código para calcular el ángulo del gradiente es:

```
phi = np.arctan2(iy, ix) # ángulos en  $[-\pi, \pi]$ 
phi[phi < 0] += np.pi # convertimos negativos a  $[0, \pi]$ 
phi_grados = np.degrees(phi) # convierte a grados: 0 a
    180

# 4) se discretizan los angulos
angulo = discretizar_angulos(phi_grados)
```



Luego, la discretización se hace con la siguiente función:

```
def discretizar_angulos(phi_grados: np.ndarray) -> np.  
    ndarray:  
    angulo = np.zeros_like(phi_grados)  
  
    mask = (phi_grados <= 22.5) | (phi_grados > 157.5) # Zona  
        amarilla  
    angulo[mask] = 0  
    mask = (phi_grados > 22.5) & (phi_grados <= 67.5) # Zona  
        verde  
    angulo[mask] = 45  
    mask = (phi_grados > 67.5) & (phi_grados <= 112.5) # Zona  
        azul  
    angulo[mask] = 90  
    mask = (phi_grados > 112.5) & (phi_grados <= 157.5) # Zona  
        roja  
    angulo[mask] = 135  
  
    return angulo
```



Código para aplicar la supresión de no máximos:

```
m, n = magnitud.shape
magnitud_snm = magnitud.copy()

for i in range(1, m-1):
    for j in range(1, n-1):
        if magnitud_snm[i, j] == 0:
            continue
        vecinos = obtener_vecinos(magnitud, i, j, angulo[i, j])
        if magnitud_snm[i, j] < max(vecinos):
            magnitud_snm[i, j] = 0
```



Los vecinos se obtienen como:

```
def obtener_vecinos(magnitud: np.ndarray, i: int, j: int,
                    angulo: float) -> np.ndarray:
    if angulo == 0:
        return [magnitud[i-1, j], magnitud[i+1, j]]
    elif angulo == 45:
        return [magnitud[i-1, j-1], magnitud[i+1, j+1]]
    elif angulo == 90:
        return [magnitud[i, j-1], magnitud[i, j+1]]
    elif angulo == 135:
        return [magnitud[i-1, j+1], magnitud[i+1, j-1]]
    else:
        return []
```



Finalmente se aplica la umbralización con histéresis:

```
def aplicar_umbralizacion_histeresis(imagen_np: np.ndarray
    , t1: int, t2: int) -> np.ndarray:
    m, n = imagen_np.shape
    bordes = np.zeros((m, n), dtype=np.uint8)
    bordes[imagen_np > t2] = 255
    bordes[(imagen_np >= t1) & (imagen_np <= t2)] = 128
    while True:
        pixeles_promovidos = 0
        for i in range(1, m - 1):
            for j in range(1, n - 1):
                if bordes[i, j] == 128: # Si es un píxel débil
                    vecindad = bordes[i-1 : i+2, j-1 : j+2]
                    if np.any(vecindad == 255): # Si toca px fuerte
                        bordes[i, j] = 255
                        pixeles_promovidos += 1
            if pixeles_promovidos == 0:
                break
    bordes[bordes == 128] = 0
    return bordes
```


Al aplicar el **Detector de Canny** ($t_1 = 50, t_2 = 150$) sobre la imagen de la Figura 1, se obtuvo las imagen de borde visible en la Figura 2.



Figura 1: Original



Figura 2: Imagen de borde

(*) Comentario.

Sobre la misma imagen original se aplicó el **Detector de Canny** a imágenes afectadas por distintos tipos de ruido:

- Figuras 3 y 4: ruido **Gaussiano** ($\sigma = 20$, $\% = 40$) con $t_1 = 50$, $t_2 = 150$.
- Figuras 5 y 6: ruido **Sal y Pimienta** ($p = 10$) con $t_1 = 40$, $t_2 = 220$.



Figura 3: Ruido Gaussiano

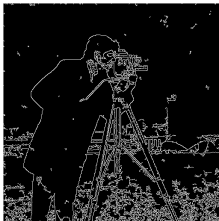


Figura 4: Canny (Gauss)



Figura 5: Ruido SyP

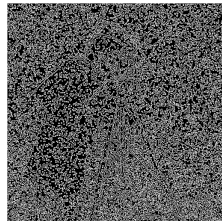


Figura 6: Canny (SyP)

Consigna 2:

Implementar el Método de Smallest Univariate Assimilating Nucleus (SU-SAN) para:

- 1 Detección de bordes.
- 2 Detección de esquinas.

Aplicarlos a la imagen Test y a su versión contaminada.



Método de SUSAN

- 1 Ubicar una máscara circular alrededor de cada pixel.
- 2 Calcular la cantidad de pixeles dentro de la máscara que tienen el mismo nivel de gris que el núcleo, salvo un umbral.

$$c(r, r_0) = \begin{cases} 1, & \text{si } |I(r) - I(r_0)| < t, \\ 0, & \text{si } |I(r) - I(r_0)| \geq t. \end{cases}$$

- 3 Luego se calculan $n(r_0)$ y $s(r_0)$.
- 4 Finalmente:

$$r_0 = \begin{cases} \text{nada,} & \text{si } s(r_0) \approx 0, \\ \text{es un borde,} & \text{si } s(r_0) \approx 0,5, \\ \text{es esquina,} & \text{si } s(r_0) \approx 0,75. \end{cases}$$

(*) c.



Para cada pixel (i, j) se realiza:

```
region = imagen_padded[i:i+k, j:j+1, :]
region_circular = region[filtro == 1]
centro = region[pad_h, pad_w, :]

dif = np.linalg.norm(region_circular - centro, axis=1)
c_r0 = (dif < t).astype(int)

n_r0 = np.sum(c_r0)
s_r0 = 1 - (n_r0 / 37)

if s_r0 < 0.35:
    continue # no borde
elif s_r0 > 0.35 and modo == "borde":
    imagen_filtrada[i, j, 2] = 255 # borde color azul
elif s_r0 > 0.60 and modo == "esquina":
    imagen_filtrada[i, j, 0] = 255 # esquina color rojo
```

(*) Se realizó un padding.

Al aplicar el **Método de SUSAN** sobre la imagen Test de la Figura 7, se obtuvieron las imágenes con bordes y esquinas resaltados visibles en la Figura 8 y Figura 9.

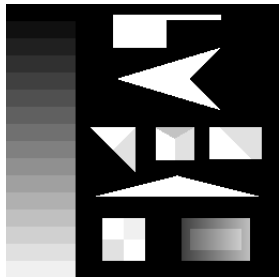


Figura 7: Original

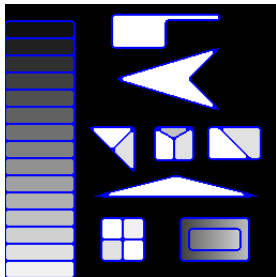


Figura 8: Bordes

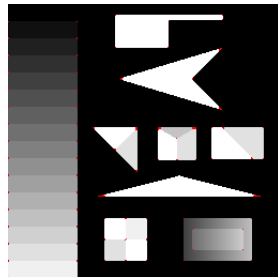


Figura 9: Esquinas

(*) c.

Al aplicar el **Método de SUSAN** sobre la imagen Test de la Figura 10 con ruido Gaussiano ($\% = 40, \sigma = 20$), se obtuvieron las imágenes con bordes y esquinas resaltados visibles en la Figura 11 y Figura 12.

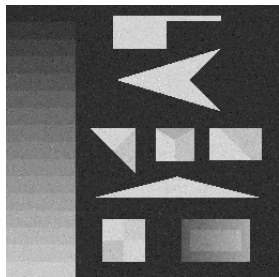


Figura 10: Ruido Gauss

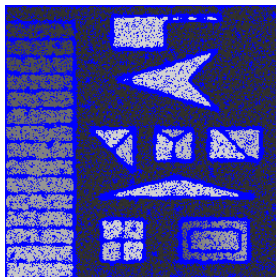


Figura 11: Bordes

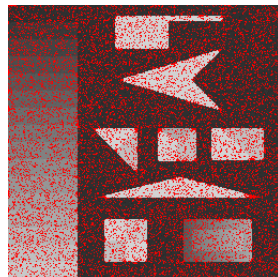


Figura 12: Esquinas

(*) c.

Consigna 3:

Implementar la transformada de Hough para detectar rectas y aplicarla a la imagen Test y a su versión contaminada.



La Transformada de Hough consiste en encontrar formas parametrizables (rectas, círculos, elipses) dentro de una imagen.

Transformada de Hough

- 1 Hallar los bordes la imagen utilizando un método de detección de bordes.
- 2 Umbralizar la imagen para obtener una imagen binaria.
- 3 Subdividir el plano (r, θ) discretizando en una cantidad específica de puntos.
- 4 Para cada pixel blanco de la imagen, decidir si cumple la ecuación normal de la recta, en caso afirmativo aumentar el acumulador.
- 5 Examinar el contenido de las celdas del acumulador con altas concentraciones (tomar el máximo o umbralizar).
- 6 Graficar las rectas encontradas.



El código para subdividir el plano es:

```
m, n = magnitud.shape
diagonal = int(np.round(np.sqrt(m**2 + n**2)))
r = np.arange(-diagonal, diagonal)

theta = np.linspace(-np.pi/2, np.pi/2, 180) # valores entre
      +-90
cos_theta = np.cos(theta)
sin_theta = np.sin(theta)
```



Luego se calculan todas las rectas y se suma al acumulador cuando corresponde.

```
A = np.zeros((len(r), len( $\theta$ )), dtype=np.int32)
 $\epsilon$  = 2
y, x = np.where(magnitud == 255)

prod = x[:, None] * cos_ $\theta$ [None, :] + y[:, None] * sin_ $\theta$ [
    None, :]      # (P,  $\theta$ )

for i, r_i in enumerate(r):
    dif = np.abs(r_i - prod)
    A[i, :] = np.sum(dif <  $\epsilon$ , axis=0)
```

(*) No realicé ambas operaciones de manera vectorizada ya que ocupaba demasiado lugar en memoria.



Con las celdas del acumulador computadas procedemos a examinarlas, en este caso dado un umbral:

```
maximos = np.argwhere(A >= umbral)
print(f"Lineas encontradas: {maximos.shape[0]}")
```



Finalmente se dibujan las rectas correspondientes:

```
resultado_np = dibujar_rectas(imagen_np, maximos, r,  $\theta$ )  
  
return resultado_np
```

Sobre la misma imagen original se aplicó la **Transformada de Hough** para la detección de líneas, considerando dos casos: la imagen original y la imagen afectada por ruido **Sal y Pimienta**.

- Figuras 13 y 14: imagen original con **umbral = 200**.
- Figuras 15 y 16: imagen con ruido **Sal y Pimienta** ($p = 5$) con **umbral = 350**.

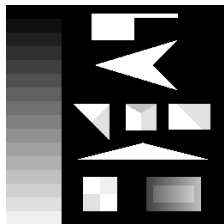


Figura 13: Imagen original

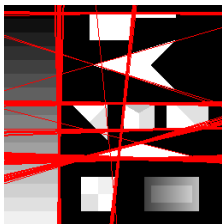


Figura 14: Hough (umbral = 200)

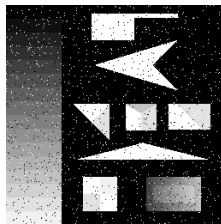


Figura 15: Ruido SyP ($p = 5$)

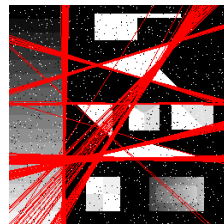


Figura 16: Hough (umbral = 350)

Consigna 4:

Implementar el método de segmentación basado en conjuntos de nivel e intercambio de pixels y aplicarlo a imágenes estáticas. Aplicarlo también a imágenes contaminadas con ruido. Analizar para que tipo de imágenes es conveniente utilizar el método.



El **Método de Intercambio de Píxeles** realiza una segmentación iterativa basada en diferencias de intensidad entre regiones vecinas.

En cada iteración:

- 1 Se calculan los parámetros θ_0 y θ_1 del fondo y el objeto (la media de sus intensidades).
- 2 Se identifican los bordes de la región del objeto.
- 3 Para cada píxel en L_{out} , evaluar la función de decisión $F_d(x)$. Si $F_d > 0$, el píxel pasa a la región interior.
- 4 Para cada píxel en L_{in} , evaluar $F_d(x)$. Si $F_d \leq 0$, el píxel se mueve fuera de la región.



```
def obtener_segmentacion_cn_ip(imagen_np: np.ndarray,
    region1: np.ndarray) -> np.ndarray:
    region0 = ~region1

     $\theta_0$  = np.mean(imagen_np[region0], axis=0)
     $\theta_1$  = np.mean(imagen_np[region1], axis=0)

    L_out, L_in = encontrar_bordes(region1)

    region1_nueva = region1.copy()

    Fd_L_out = Fd( $\theta_0$ ,  $\theta_1$ , imagen_np[L_out])
    region1_nueva[L_out] = Fd_L_out > 0 # Out = False

    Fd_L_in = Fd( $\theta_0$ ,  $\theta_1$ , imagen_np[L_in])
    region1_nueva[L_in] = Fd_L_in > 0 # In = True

    return region1_nueva
```

Se aplicó el **Método de Intercambio de Píxeles** sobre la imagen de la Figura 17, realizando dos segmentaciones distintas para resaltar diferentes regiones de la imagen.

- Figura 17: imagen original utilizada.
- Figura 18: primera segmentación obtenida.
- Figura 19: segunda segmentación, enfocada en una región distinta.

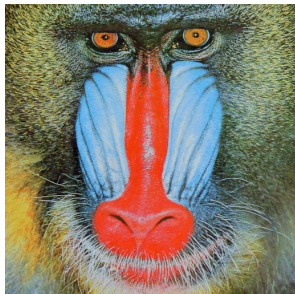


Figura 17: Imagen original

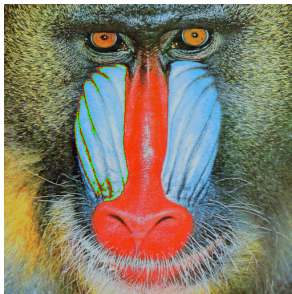


Figura 18: Segm. 1

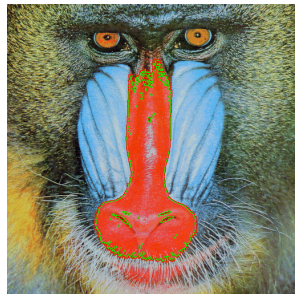


Figura 19: Segm. 2

Se aplicó el **Método de Intercambio de Píxeles** sobre la imagen *primate.png* contaminada con **ruido Gaussiano** ($\sigma = 60$) afectando el 30 % de los píxeles, realizando dos segmentaciones distintas para resaltar diferentes regiones bajo condiciones de ruido.

- Figura 20: imagen contaminada con ruido Gaussiano.
- Figura 21: primera segmentación obtenida.
- Figura 22: segunda segmentación, enfocada en una región distinta.

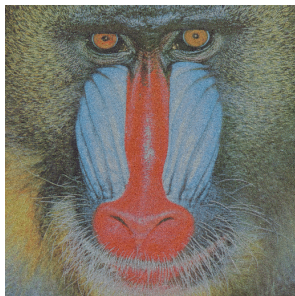


Figura 20: Ruido

Matías Cisnero

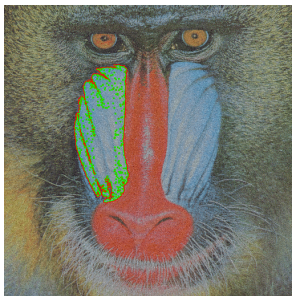


Figura 21: Segm. 1

Trabajo Práctico 3 Procesamiento de Imágenes

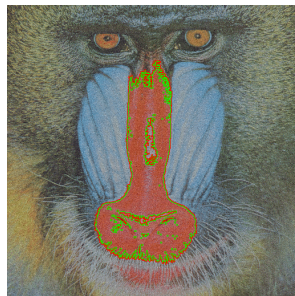


Figura 22: Segm. 2

6 de Noviembre de 2025

27 / 30



Todo el código y los resultados de este trabajo están disponibles en mi repositorio de GitHub:

https://github.com/matias-cisnero/procesamiento_imagenes



NumPy Documentation: Broadcasting.

<https://numpy.org/doc/stable/user/basics.broadcasting.html>



Imagen "*Leucanthemum vulgare Blüte*" por Mariofan13.

Wikimedia Commons. Licencia *CC BY-SA 3.0 Unported*.

[Ver imagen en Wikimedia Commons](#)



Gonzalez, R. and Woods, R. (2018). Digital Image Processing. Pearson Education 4th Edition, New York.

¡Gracias!

